EVIDENCE-FIRST PORTFOLIO

Voice interface and spoken tool use

Individual printable topic report.

Generated 2026-06-17

Why I built it

Voice control is powerful because it lowers friction, but spoken commands are ambiguous and can trigger tools. The system needed tool routing, muting, fallbacks, destructive-command filtering, and a privacy-forward plan.

Architecture and evidence

`voice.py` contains Gemini Live config, sample-rate handling, tool declarations, dispatcher, vault tools, deep-think route, shell guard, reconnect/fallback loop, and VoiceBridge experiments. Planning notes define latency targets, privacy red lines, and acceptance criteria.

Claim boundary: Wake-word/macOS app features should be described as planned or partially integrated unless retested.

- Python terminal voice interface streams microphone audio into Gemini Live.
- The Live config exposes tools for vault search/read/write, deep thinking through Miguel Core, brain status, context, and constrained shell execution.
- Dangerous shell patterns are blocked and command execution is timeout-limited.
- A later design adds wake-word, local STT, hybrid TTS, barge-in, and privacy boundaries.

What I can show with this

- Voice was integrated with tools, not just used as speech-to-chat.
- The interface can route from spoken input into vault operations, daemon calls, and constrained shell execution.
- The design includes muting, fallbacks, destructive-command blocking, and privacy plans.
- The later pipeline notes show awareness of wake-word, local STT, TTS, and barge-in constraints.
- Voice commands should be treated as high-friction-to-confirm, low-friction-to-issue.
- Always-listening systems require local processing and explicit privacy boundaries.
- Barge-in is a safety feature because it lets the user stop the system quickly.
- A beautiful voice is irrelevant if authority and privacy are unclear.
- Retest browser/microphone E2E with a clean public demo.
- Separate safe voice tools from confirmation-required tools.
- Add transcript-level audit and user correction paths.
- Measure latency rather than relying on target budgets.